NATIONAL AUTONOMOUS UNIVERSITY OF MEXICO

FACULTY OF ENGINEERING

COMPUTER GRAPHICS AND HUMAN-COMPUTER INTERACTION

SEMESTER 2027 - 1

TEXTURING, COLOR, AND LIGHTING

STUDENTS	*Basilio Illescas Andrés*	*Product Owner*
	Trejo Olvera Emmanuel	*Scrum Master*
	Pérez Olvera Alexis Abraham	*Developer*
	Cruz Macedo Samuel Santiago	*Developer*
PROFESSOR	*Eng. José Ramón Pérez Athié*	
GROUP	*04*	

Ciudad Universitaria, CDMX

October 28, 2026

Contents

1 Introduction **2**

2 Objectives **2**

2.1 General Objective 2

2.2 Specific Objectives 2

3 Scope of the Delivery **2**

4 Agile Workflow Methodology **3**

5 Design and Development **3**

5.1 Blender Workflow: Procedural Textures 3

5.2 Normal Mapping and PBR Materials 3

5.3 Procedural Generation of UVs and Alpha Clipping 4

5.4 Hybrid Lighting and Transmission 4

5.5 Atmospheric Environment (Skybox) 4

6 Architectural Challenges and Cross-Platform Compatibility **4**

7 Project Structure **5**

8 Application Screenshots **7**

9 Compilation and Execution **10**

9.1 Launchers by Platform 10

9.1.1 macOS 10

9.1.2 Windows 10

9.1.3 Linux 11

9.2 Logs and Diagnostics 11

10 Results and Conclusions **11**

11 References **12**

1 Introduction

After consolidating the geometric modeling base in the previous stage, the project advances towards the application of optical and lighting properties to bring realism to the Day of the Dead scene. It should be noted that most of our custom models were remodeled for this phase, demonstrating a greater commitment to the project and significantly increasing the visual and geometric quality of this delivery.

In this stage, the gray models are transformed through the application of texture maps (Base Color, Roughness, and Normals), the configuration of transparent materials, the creation of an immersive environment (*Skybox*), and the implementation of a dynamic lighting system.

Development was maintained in C++17 using OpenGL 3.3. Specific *shaders* were programmed to process simplified PBR textures and light the scene using a hybrid model that combines ambient and directional light (Moon) with point lights (candles and lanterns) and quadratic attenuation. All the work was structured to maintain optimal performance while processing thousands of textured instances simultaneously.

2 Objectives

2.1 General Objective

Implement a real-time texturing, materials, and lighting system over the previously modeled scene, achieving an immersive visual finish that respects geometric proportions and optimizes resources in a cross-platform environment.

2.2 Specific Objectives

- Design and bake PBR textures (Color, Normals, and Roughness) from procedural nodes in Blender.
- Implement dynamic 2D image loading with ImageIO on macOS and `stb_image` on Windows and Linux.
- Configure a *Skybox* with panoramic textures to simulate the lighting transition of a sunset.
- Develop *shaders* that calculate point lights with attenuation and simulated optical refraction for glass and liquids.
- Guarantee correct compilation and execution on macOS, Windows, and Linux distributions with OpenGL 3.3 support.

3 Scope of the Delivery

The scope of this phase includes texturing the custom altar and environment models, applying materials to library models, procedurally creating UV coordinates, transparency mapping (alpha clipping), and programming the lighting *shaders*.

This delivery includes diffuse maps, normal maps calculated in real-time through mathematical derivatives of the fragments, metalness/roughness channels, and the cubic atmospheric environment. It also

incorporates simulated optical refraction through transmittance calculations and the *Fresnel* effect for vases and glasses.

Complex skeletal animations, high-resolution dynamic shadows (*Shadow Mapping*), and real-time ray-traced specular reflections remain outside the scope of this delivery.

4 Agile Workflow Methodology

To manage this second stage of the project, the team continued iterating on the *Scrum* framework, adapting the *sprints* to the specific needs of UV mapping, *shader* programming, and performance testing.

The *Product Backlog* was restructured to focus on the visual finish and optimization. Development was divided into four main iterations (*sprints*):

1. **Definition and Procedural Design:** Assignment of material properties, creation of node networks in Blender, and baking of PBR textures.
2. **Graphics Programming:** Development and integration of *shaders* for hybrid lighting, real-time normal calculations, and optical transmission.
3. **Integration and Cross-Platform Testing:** Creation of native routines for image loading, *Skybox* validation, and resolution of deployment conflicts on macOS, Linux, and Windows.
4. **Optimization and Refactoring:** Identification of lighting bottlenecks, geometric retopology of models with excessive polygon counts, and automation via native launchers.

Asynchronous sync meetings allowed us to early identify architectural roadblocks (such as Git repository conflicts due to OS-specific configurations), facilitating quick decision-making by the *Scrum Master* and the developers to unify the `CMakeLists.txt` and keep the delivery within the established deadlines.

5 Design and Development

5.1 Blender Workflow: Procedural Textures

To guarantee high visual quality without relying on external photographs, a procedural workflow was chosen. The textures of our custom models were designed directly in Blender's node editor using mathematical noise functions and color blending.

Subsequently, these complex node networks were baked into traditional bitmaps (Color, Roughness, and Normals). Finally, the models, along with their textures, were exported under the glTF 2.0 standard and stored in the `assets/models/propios/` path. This packaging ensured highly efficient reading when loading the scene in OpenGL.

5.2 Normal Mapping and PBR Materials

Instead of loading and precalculating tangent vectors in the vertices, the *fragment shader* (`escena.frag`) calculates the mapped normals in real-time. This was achieved by using the `dFdx` and `dFdy` functions

over the vertex positions and UV coordinates, allowing us to extract highly detailed reliefs for elements such as the *pan de muerto*, adobe walls, and calaveritas in a highly optimized manner.

5.3 Procedural Generation of UVs and Alpha Clipping

For the cobblestone path and the cempasuchil petal carpet, UV coordinates were generated procedurally in the `Camino.cpp` file based on the arc length of the Catmull-Rom spline. The *shader* processes masks with alpha clipping (`uCorteAlfa`) to shape the jagged edges of the *papel picado* and discard transparent pixels from the scattered petals, drastically reducing the geometric load.

To avoid geometric overlapping or clipping of complex textures, a spatial height grid was implemented in the `ApoyoDulce` class (`ApoyoDulces.h`), which clips each triangle against the corresponding cell, guaranteeing a perfect and natural stacking of the elements.

5.4 Hybrid Lighting and Transmission

The lighting model abandoned the generic directional light from the modeling phase in favor of a hybrid system. This consists of:

- **Directional moonlight:** A faint directional tint modified with ambient occlusion parameters.
- **Point lights:** An array in the *shader* that supports up to 64 simultaneous sources (applied to candles and lanterns) with quadratic attenuation based on a fixed radius.
- **Optical transmission:** An approximate calculation without screen-space refraction, utilizing the index of refraction (`uIor`) and the *Fresnel* calculation to add realism to glass and water objects, reinforcing edges and allowing visibility backward without artifacts.

5.5 Atmospheric Environment (Skybox)

A cubemap for the sky was integrated via the `Skybox` class. The sky's *vertex shader* removes the translation of the view matrix, ensuring the horizon remains static as the player moves. In the *fragment shader* (`cielo.frag`), the pixel's luminance (*luma*) is extracted to dynamically darken the original sunset and blend it into an immersive nocturnal tonality.

6 Architectural Challenges and Cross-Platform Compatibility

Validation was performed on macOS, Windows, and Linux. In addition to visual behavior, the team tested CMake configuration, compilers, resource discovery, and launchers without visible consoles. The relevant incidents and their final solutions were:

1. **Generated artifacts and diverging Git histories.** CMake, Xcode, and Visual Studio products are not portable between computers. The `.gitignore` file was expanded to exclude `build-*`, caches, binaries, objects, libraries, logs, and operating-system metadata. Documentation and sources remain versioned; diverging histories were integrated with `git pull –rebase` so remote documentation and local fixes were both preserved.

2. ***Framebuffer* size on high-resolution displays.** On Retina and other scaled displays, logical window dimensions do not match renderable pixels. The application now calls `glfwGetFramebufferSize` before `glViewport`, preventing the scene from occupying only part of the window.

3. **Portable resource discovery.** The application must not depend on the directory from which it was launched. `Rutas.cpp` obtains the actual executable location through `GetModuleFileNameW` on Windows and `/proc/self/exe` on Linux, then searches for `assets/` and `shaders/` through `std::filesystem`. Wide paths are used on Windows for UTF–8 filenames. This removed dependencies on particular separators and working directories.

4. **Textures visible on macOS but missing on Windows and Linux.** The initial `ImageLoader.cpp` implemented decoding only inside `#ifdef __APPLE__`; every other platform returned `false`. Geometry and flat colors appeared, while PNG, JPEG, and the skybox failed. The final solution retains `ImageIO/CoreGraphics` on macOS and integrates `stb_image` 2.30 on Windows and Linux. Its header is vendored under `external/stb/`, with only PNG and JPEG decoders enabled. CoreGraphics alpha un-premultiplication remains in place to avoid dark cutout edges on macOS.

5. **macOS launcher.** Using a shell script directly as `CFBundleExecutable` caused launch failures and Finder-cache inconsistencies. The final bundle uses a small native Mach–O launcher that locates the project and delegates the build to `Contents/Resources/iniciar.zsh`. The local academic application is unsigned, so its first launch may require right-clicking and selecting **Open**. Details are written to `logs/lanzador-macos.log`.

6. **VBScript, PowerShell, and Visual Studio on Windows.** The VBScript command used to start PowerShell originally contained an unterminated string; it was repaired by constructing quotes with `Chr(34)`. The launcher then searched only for Visual Studio 2022 while the test computer had Visual Studio 2026. It now queries `vswhere.exe`, automatically selects `Visual Studio 17 2022` or `Visual Studio 18 2026`, and uses separate build directories.

7. **Truncated diagnostics and concurrent Windows builds.** With `ErrorActionPreference=Stop`, Windows PowerShell treated the first line written by CMake to `stderr` as an exception and truncated the useful diagnostic. Native invocations are now wrapped so all output is recorded and `LASTEXITCODE` is evaluated. Furthermore, double-clicking during a hidden build started a second MSBuild process and produced *Permission denied* errors on `.obj` files; a named mutex now prevents concurrent runs.

8. **Compiler and floating-point differences.** MSVC rejected the indirect availability of `std::array`, so `¡array¿` was explicitly included. Windows also produced 3055 instances against a macOS reference of 3057 because of floating-point rounding in geometric tests. A small deviation is now logged as a warning, while a larger difference still stops loading so genuine import errors are not hidden.

9. **OpenGL 3.3 inside a virtual machine.** Windows 11 ARM under UTM compiled successfully, but its VirtIO 3D driver exposed DirectX acceleration rather than a compatible OpenGL 3.3 context. Window creation failed, whereas the same repository ran on physical Windows with the official GPU driver. A successful VM build therefore does not guarantee the required OpenGL level; guest graphics capabilities must be verified independently.

7 Project Structure

The project incorporates new files and folders intended for the administration of textures and updated *shaders*.

Table 1: New or updated files and folders.

Element	Purpose
`src/render/Skybox.cpp`	Cubemap loading, *luma* manipulation, and spherical sky rendering.
`src/render/ImageLoader.cpp`	PNG/JPEG decoding through ImageIO on macOS and `stb_image` on Windows/Linux.
`src/Rutas.cpp`	Executable discovery and portable resolution of `assets/` and `shaders/`.
`src/render/Shader.cpp`	Syntactic parser for macro injection (`#define`) and dynamic GLSL compilation.
`shaders/escena.frag`	Lighting algorithms, partial derivatives for reliefs, transmittance, and PBR variables.
`shaders/escena.vert`	Spatial transformation adapted for processing textured instances and color tints.
`assets/textures/`	Base textures, color maps, and metalness/roughness generated procedurally.
`assets/skybox/atardecer/`	Spherical image fragments to process the panoramic environment of the scene.
`external/stb/stb_image.h`	Vendored single-header image decoder for Windows and Linux.
`ejecutables/`	Silent launchers and build scripts for macOS, Windows, and Linux.

8 Application Screenshots



Figure 1: General view of the illuminated altar.

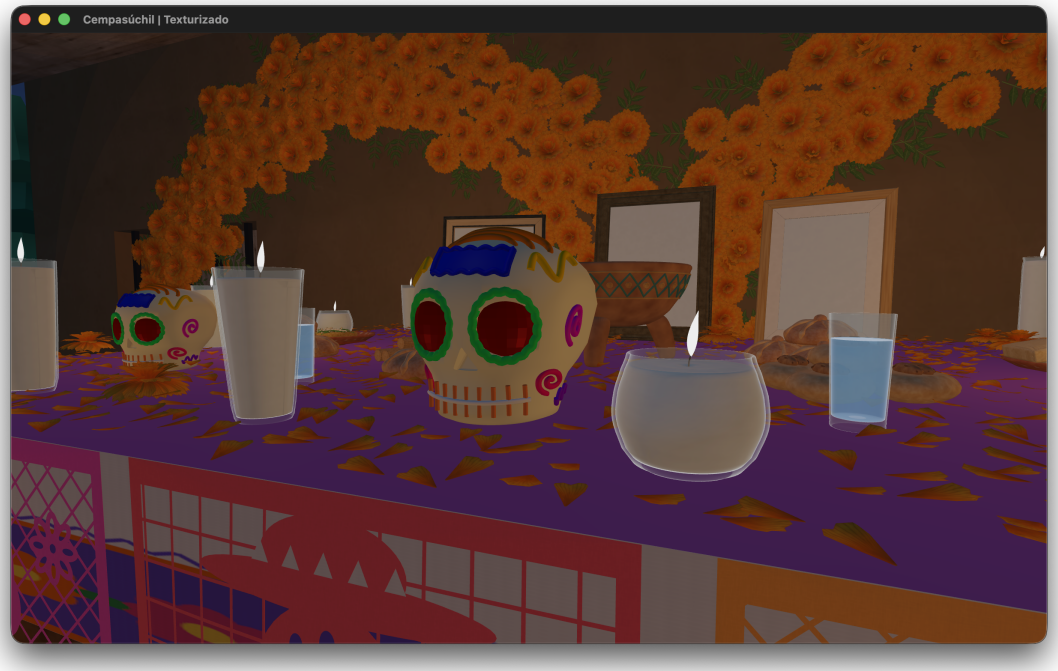


Figure 2: Detail of PBR textures and point light effect on objects.

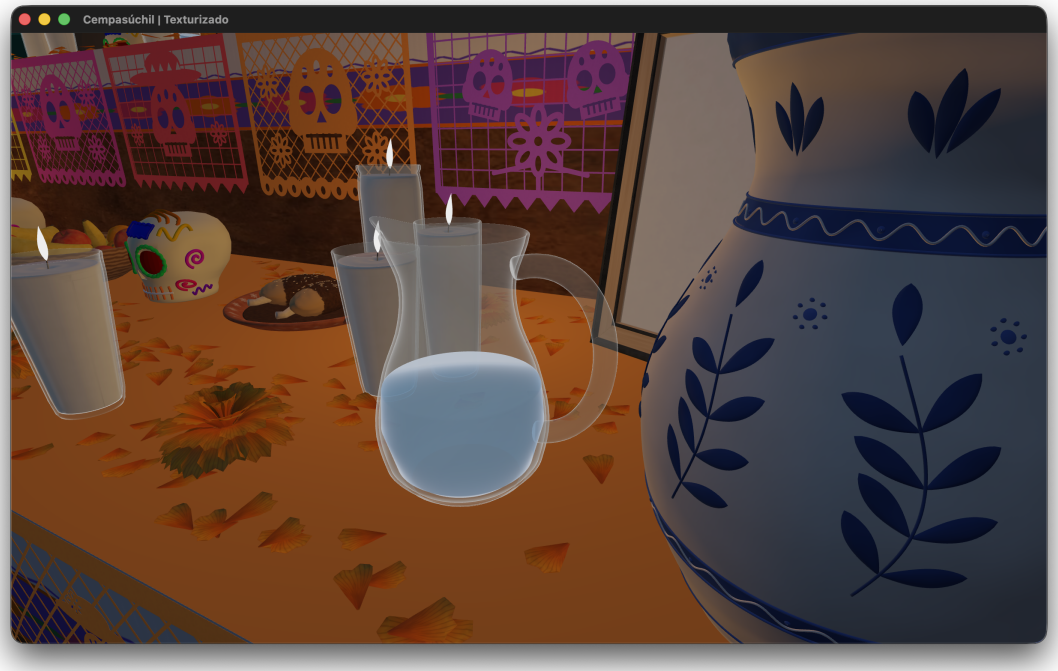


Figure 3: Optical approximation in glass and liquid materials.

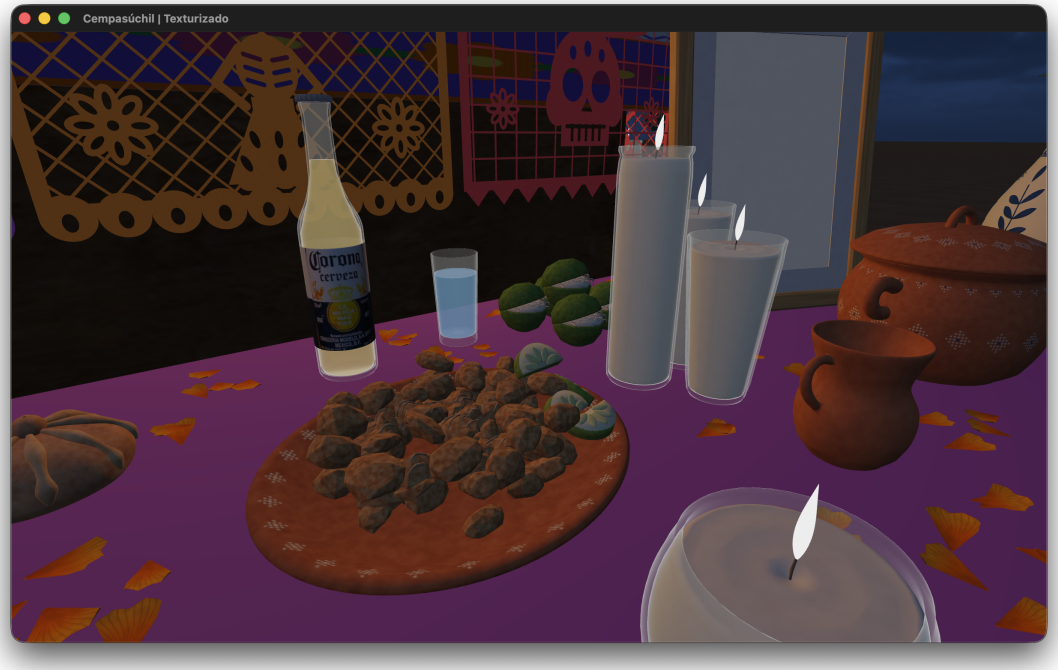


Figure 4: Optical approximation in glass and liquid materials.

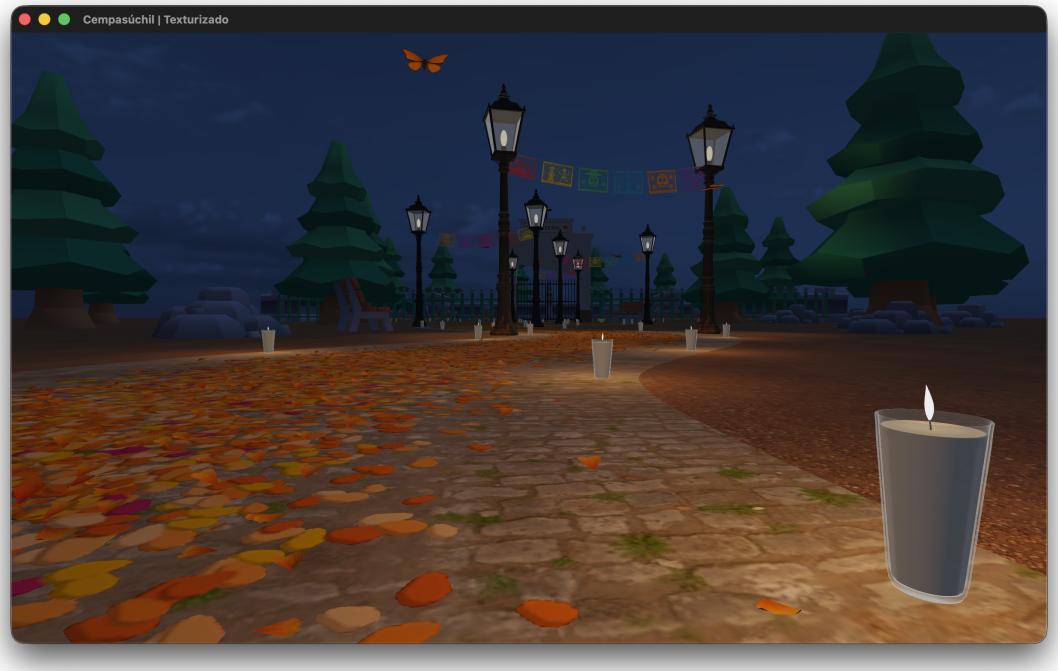


Figure 5: Procedural cobblestone path illuminated by lanterns and candles.

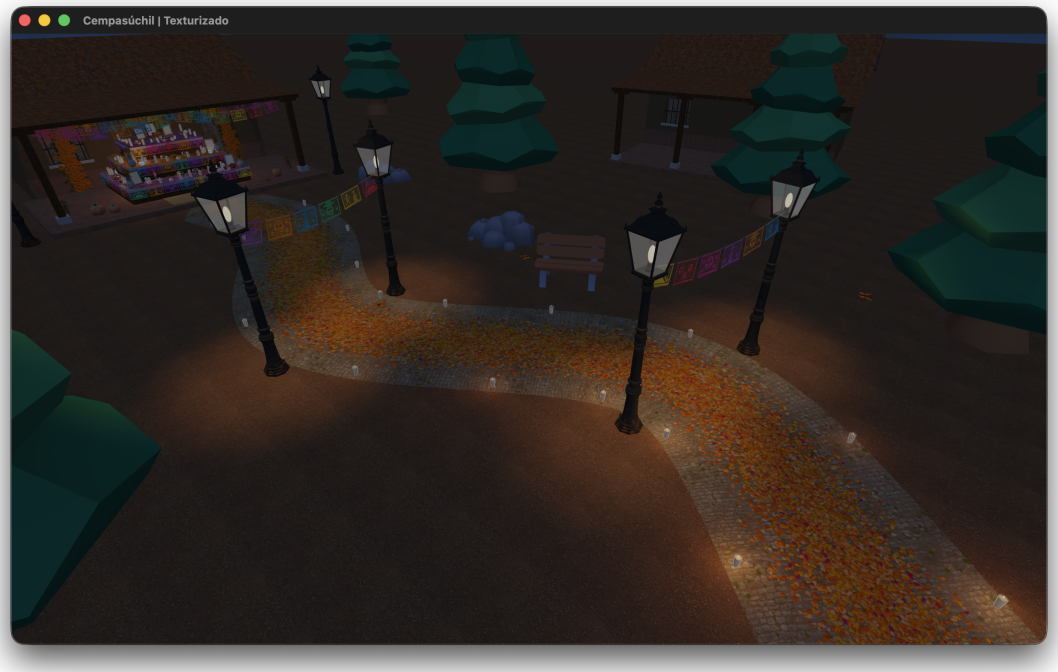


Figure 6: General view of the illuminated path.

9 Compilation and Execution

Execution was validated on macOS, physical Windows, and Linux. Every platform requires a GPU or driver that exposes OpenGL 3.3; 3D or DirectX acceleration alone does not satisfy this requirement.

To optimize the workflow and facilitate project review, the team developed native launchers for each operating system. These shortcuts completely automate the build and deployment cycle, standardizing the process under the following features:

- Configure and compile the project via CMake strictly in `Release` mode.
- Update and synchronize the resource directories (`assets` and `shaders`) into the output directory.
- Automatically execute the scene once compilation is finished without running terminal processes.
- Keep the console hidden during the process to offer a clean graphical experience.
- Redirect and write all technical output, warnings, and diagnostics into the `logs/` folder located at the root of the project.

9.1 Launchers by Platform

The repository must reside in a local writable directory because each launcher creates a `build-*` directory. Launchers do not download dependencies and are stored under `ejecutables/`.

9.1.1 macOS

CMake, Xcode Command Line Tools, and system OpenGL 3.3 support are required. After cloning or downloading the repository, open:

```
ejecutables/macOS/TexturizadoCempasuchil-0.2.1.app
```

Because this local academic application is unsigned, Gatekeeper may block the first attempt; right-click it and select **Open**. The launcher searches for CMake in PATH, Homebrew, and the official CMake application, builds under `build-mac-app/`, and retains the last valid binary if a later rebuild fails.

9.1.2 Windows

Requirements are 64-bit Windows, a GPU driver with OpenGL 3.3, CMake in PATH, and Visual Studio Community 2022 or 2026 with the **Desktop development with C++** workload, x64/x86 MSVC tools, and a Windows SDK. Then open:

```
ejecutables“windows“TexturizadoCempasuchil.vbs
```

VBScript starts PowerShell without a console; `vswhere` selects the installed generator and targets x64 under `build-windows-vs17/` or `build-windows-vs18/`. Windows ARM64 can cross-compile the x64 executable, but execution still requires a compatible OpenGL driver.

9.1.3 Linux

CMake, a C++17 compiler, OpenGL, and the X11 development packages used by GLFW are required. Debian/Ubuntu-based distributions can prepare the environment with:

```
sudo apt update
sudo apt install build-essential cmake libgl1-mesa-dev “
   xorg-dev
```

On Arch Linux use the equivalent `base-devel`, `cmake`, `mesa`, and `libx11` packages. Mark the `.desktop` file as trusted/executable and open it, or run:

```
chmod +x ejecutables/linux/iniciar˙linux.sh
./ejecutables/linux/iniciar˙linux.sh
```

9.2 Logs and Diagnostics

The platform logs are `logs/lanzador-macos.log`, `logs/lanzador-windows.log`, and `logs/lanzador-linux.log`. Internal scene messages are stored as `registro.txt` in the build directory. If the application reports that it cannot create a window, check the OpenGL version and update the vendor GPU driver; reinstalling the compiler does not repair a graphics limitation.

NOTE: For further details on the internal workings of these automated launchers, to review the exact dependencies of each system, or to check the location of the log and diagnostic files, it is highly recommended to read the `README.md` file located at the `ejecutables/README.md` path.

10 Results and Conclusions

The texturing and lighting phase significantly elevated the visual impact and immersion of the Day of the Dead environment. However, implementing the hybrid lighting model presented a major performance challenge for the team. By integrating dynamic light sources, we detected severe bottlenecks because some models exported from Blender had an excessive polygon density; the graphics engine was calculating lighting per fragment over an unnecessary amount of triangles, saturating the processing pipeline. The implemented solution was to optimize these models, drastically reducing their triangle count (retopology and decimation) while retaining a visual appearance practically identical to the original thanks to the detail provided by the normal maps.

The use of procedurally crafted PBR textures, coupled with normal calculations derived directly in the *fragment shader*, demonstrated an excellent balance between photographic quality and algorithmic optimization. Likewise, the cross-platform architectural design was crucial in successfully overcoming technical barriers related to native libraries, file systems, repositories, and high-resolution interfaces.

Proper alpha mapping to reduce polygons (as seen in the petal carpet), geometric optimization of the meshes, and the simulated refraction system allow us to affirm that the delivery satisfactorily meets the proposed visual parameters. For the final phase, the robust and optimized technical environment built so far will serve as an ideal foundation to seamlessly integrate particle systems and animations.

11 References

- Blender Foundation. (2026). *Blender 5.2*. https://www.blender.org/
- Khronos Group. (2021). *glTF 2.0 Specification*. https://registry.khronos.org/glTF/specs/2.0/glTF-2.0.html
- Khronos Group. (2010). *OpenGL 3.3 Core Profile Specification*. https://registry.khronos.org/OpenGL/specs/gl/glspec33.core.pdf
- Kitware. (2026). *CMake Documentation*. https://cmake.org/documentation/
- nothings/stb. (2025). *stb_image 2.30*. https://github.com/nothings/stb
- GLFW. (2026). *GLFW Documentation*. https://www.glfw.org/documentation.html